MEMORY DESIGN Using Verilog

ARNAV BANSAL

Presented here is a memory design project using Verilog hardware description language (HDL). This project is simulated using ModelSim software, and the design is tested through a simulation process. Simulation is done to check, verify and ensure that what is wanted is what is described.

Simulation is carried out through dedicated tools. With every simulation, the results are studied to identify errors in the design description. Errors are corrected and then another simulation is carried out. Simulation and changes to design description together form a cyclic iterative process, repeated until an error-free design is evolved.

Design description is an activity independent of the target technology or manufacturer. It results in a description of the digital circuit. To translate it into a tangible circuit, one goes through the physical design process.

The same constitutes a set of activities closely linked to the manufacturer and the target technology. For designing projects in HDLs, you have to follow the very large scale integration (VLSI) design flow and verification steps required in each step.

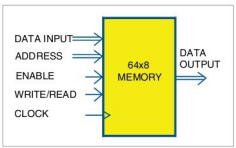

Fig. 1: Block diagram of 64x8 bit memory

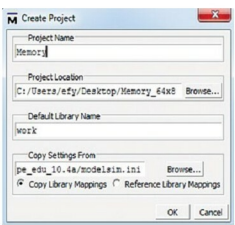

Fig. 2: Create Project window

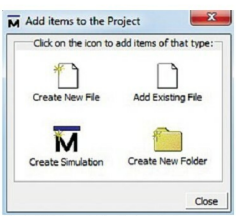

Fig. 3: Add items to the Project window

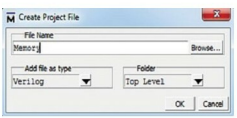

Fig. 4: Create Project File window

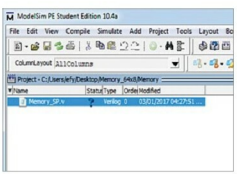

Fig. 5: Workspace window

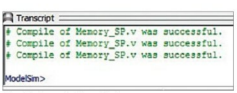

Fig. 6: Compilation window

Single-port memory

The single-port memory is basically the design as per your defined specifications. Then, as per the specified width and depth, define the memory block that can also be verified using field programmable gate array (FPGA) boards. Design hierarchy also plays an important role in designing the basic building blocks required in each step of verification.

In this project we design a 64-bit x 8-bit, which is a single-port design with common read and write addresses in Verilog.

While designing this project in ModelSim and test benches, follow the guidelines mentioned below:

1. When modelling sequential logic, use non-blocking assignments
2. When modelling latches, use non-blocking assignments
3. When modelling combinational logic with an always block, use blocking assignments
4. When modelling both sequential and combinational logic within the same always block, use non-blocking assignments
5. Do not mix blocking and non-blocking assignments in the same always block
6. Do not make assignments to the same variable from more than one always block
7. Use $strobe to display values that have been assigned using non-blocking assignments
8. Do not make assignments using #0 delays

Memory design

The memory block diagram is shown in Fig. 1. It takes a few as-